

Kamino Yvault

Security Assessment

October 26th, 2026 — Prepared by OtterSec

Tamta Topuria tamta@osec.io

Ajay Shankar Kunapareddy d1r3wolf@osec.io

Artem Astakhov aikasimirov@osec.io

Table of Contents

Executive Summary **3**

Overview 3

Key Findings 3

Scope **4**

Findings **7**

Vulnerabilities **8**

OS-KYV-ADV-00 | Theoretical-Tick Reversal Livelocks Autodrift 9

OS-KYV-ADV-01 | Zero Thresholds Bypass Snapshot Deviation Checks 10

OS-KYV-ADV-02 | Rounding Up Inflates Percentage-Based Investment Caps 11

OS-KYV-ADV-03 | Rounded Meteora Withdrawals Inflate Withdrawal Cap 12

General Findings **14**

OS-KYV-SUG-00 | Stale Pending Ticks Enable Delayed Rebalances 16

OS-KYV-SUG-01 | Unreachable Admin Authority Validation 17

OS-KYV-SUG-02 | Local Admin Restrictions Incorrectly Block Global Admin 19

OS-KYV-SUG-03 | Reward Instructions Bypass Emergency Mode Restrictions 20

OS-KYV-SUG-04 | Insufficient Validation of Invalid Input Values 21

OS-KYV-SUG-05 | Code Maturity 22

OS-KYV-SUG-06 | Reward Discount Fallback No Longer Preserves Incentive 23

OS-KYV-SUG-07 | Default Branch May Bypass Midpoint Cap 24

OS-KYV-SUG-08 | Reference Price Configuration Causes Investment Livelock 25

Appendices

Vulnerability Rating Scale **26**

© 2026 Otter Audits LLC. All Rights Reserved.

TABLE OF CONTENTS

Procedure **27**

© 2026 Otter Audits LLC. All Rights Reserved.

01 — Executive Summary

Overview

Kamino Finance engaged OtterSec to assess the `yvault` PRs. This assessment was conducted between May 22nd and October 8th, 2026. For more information on our auditing methodology, refer to Appendix B.

Key Findings

We produced 13 findings throughout this audit engagement.

In particular, we identified an issue where, when the theoretical tick reverses, `Autodrift` may select a drift direction inconsistent with the movement needed to reach the target midpoint. The resulting `DriftingOppositeDirection` error can repeatedly block rebalancing, leaving liquidity in a stale or out-of-range position (OS-KYV-ADV-00). We also found that when both strategy-level and global snapshot-price deviation thresholds are zero, invest skips deviation validation despite enforcing snapshot freshness, allowing investments at prices significantly different from the reference snapshot (OS-KYV-ADV-01).

Additionally, using `LiquidityCalculationMode::Deposit` rounds invested token amounts upward, potentially inflating the strategy's `USD` valuation and percentage-based investment cap (OS-KYV-ADV-02). Furthermore, `Meteora` withdrawals charge withdrawal caps based on rounded-up `CLMM` removal amounts rather than the user's actual redemption, overstating withdrawal activity and potentially exhausting withdrawal capacity prematurely (OS-KYV-ADV-03).

We also advised adding an expiration timestamp to pending rebalance ticks to prevent delayed execution of obsolete ranges (OS-KYV-SUG-00), exempting the global administrator from local-admin restrictions when opening liquidity positions (OS-KYV-SUG-02), and incorporating proper error-handling logic for invalid input values (OS-KYV-SUG-04). Moreover, we made recommendations to improve overall clarity, functionality, and address discrepancies (OS-KYV-SUG-05).

© 2026 Otter Audits LLC. All Rights Reserved.

02 — Scope

The source code was delivered to us in a Git repository at https://github.com/Kamino-Finance/yvaults. Audits were performed on the following PRs:

1. PR#1010
2. PR#1047
3. PR#1048
4. PR#1032
5. PR#1035
6. PR#1036
7. PR#1040
8. PR#1041
9. PR#1017
10. PR#1020
11. PR#1014
12. PR#1039
13. PR#1058
14. PR#1059
15. PR#1060
16. PR#1061
17. PR#1046
18. PR#1023
19. PR#1021
20. PR#1068
21. PR#1074

© 2026 Otter Audits LLC. All Rights Reserved.

A brief description of the PRs is as follows:

Name	Description
PR#1010	Introduces pending tick ranges for manual strategies, allowing the strategy admin to preapprove a lower and upper tick while the global actions authority executes the rebalance, enabling Squads-managed strategies to rebalance without requiring the multisig and position account to sign together.
PR#1047	Deprecates the strategy handler.
PR#1048	Adds instruction to zero out strategies padding.
PR#1032	Changes the global admin handover to two steps: the current admin proposes a new admin, and the new admin must accept before the change takes effect.
PR#1035	Adds an option to lock a self-adjusting strategy to its initial price-movement direction, so it refuses to rebalance if the price trend reverses.
PR#1036	Limits how far a self-adjusting strategy's price range can shift in a single rebalance, so sudden yield-rate spikes don't cause large jumps.
PR#1041	Removes the per-strategy authority for flash-swap rebalancing and frees up the storage it used.
PR#1017	Adds configurable limits, with safe defaults, on how far the pool price may move away from the oracle price and from a stored reference price; the bot now runs the same checks before sending transactions.
PR#1020	Adds a separate "unfreeze" role that can only undo emergency actions (exit emergency mode, restore a blocked admin), so freezing and unfreezing are held by different keys.
PR#1014	Adds an "emergency council" role, both global and per strategy, that can quickly freeze a strategy or lock out its admin without having full admin rights.

© 2026 Otter Audits LLC. All Rights Reserved.

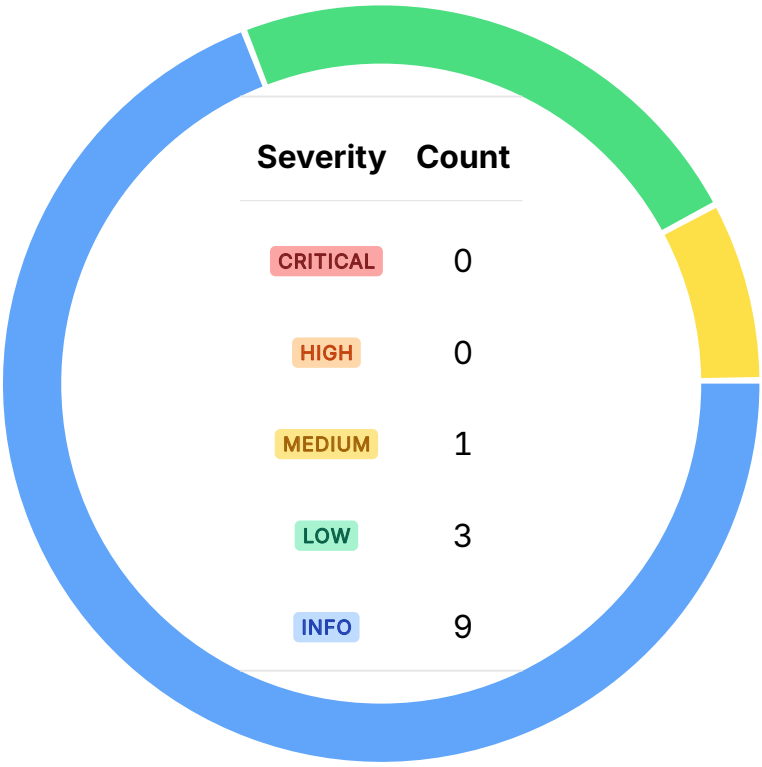
Name	Description
PR#1039	Hardens deposits: extra checks on pool price deviation and that the strategy’s token ratio matches the oracle, makes sure a manipulated pool price can’t change withdrawal value.
PR#1059	Audit follow-ups on the deposit changes: replaces silent crashes with clear errors and explicitly rejects unsupported share-minting modes.
PR#1046	Caps how far a self-adjusting strategy’s price range can move within one epoch, in addition to the existing per-rebalance cap.
PR#1023	Stores a recent price snapshot on each strategy and rejects deployments of funds when the current pool price has moved too far from that snapshot.
PR#1021	Caps how much can be deployed into a liquidity position in a single call, removes the combined deposit-and-deploy instruction, and checks the pool price against a time-averaged price before deploying.
PR#1068	Refactors price-tolerance settings to always go through getters and setters, and gives deposits their own, looser price-deviation limit so users see fewer failed deposits.
PR#1074	Measures the price snapshot’s validity window in seconds instead of blocks, adds a minimum snapshot age, allows windows up to 6 hours, and updates the bot to refresh snapshots on that schedule.

© 2026 Otter Audits LLC. All Rights Reserved.

03 — Findings

Overall, we reported 13 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



Severity	Count
CRITICAL	0
HIGH	0
MEDIUM	1
LOW	3
INFO	9

© 2026 Otter Audits LLC. All Rights Reserved.

04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities we identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in Appendix A.

ID	Severity	Status	Description
OS-KYV-ADV-00	MEDIUM	RESOLVED	When the theoretical tick reverses, `Autodrift` may select a drift direction inconsistent with the movement needed to reach the target midpoint. The resulting `DriftingOppositeDirection` error can repeatedly block rebalancing, leaving liquidity in a stale or out-of-range position.
OS-KYV-ADV-01	LOW	RESOLVED	When both strategy-level and global snapshot-price deviation thresholds are zero, `invest` skips deviation validation despite enforcing snapshot freshness. This allows investments at prices significantly different from the reference snapshot, bypassing the configured deviation caps.
OS-KYV-ADV-02	LOW	RESOLVED	Using `LiquidityCalculationMode::Deposit` rounds invested token amounts upward, potentially inflating the strategy's `USD` valuation and percentage-based investment cap.
OS-KYV-ADV-03	LOW	RESOLVED	Meteora withdrawals charge withdrawal caps based on rounded-up CLMM removal amounts rather than the user's actual redemption, overstating withdrawal activity. Repeated small withdrawals can prematurely exhaust withdrawal capacity and cause legitimate withdrawals to revert.

© 2026 Otter Audits LLC. All Rights Reserved.

Theoretical-Tick Reversal Livelocks Autodrift MEDIUM OS-KYV-ADV-00

Description

When `enforce_initial_drift_direction` is disabled and `max_ticks_per_rebalance` is nonzero, `Autodrift::get_next_rebalance_step` can reject valid rebalances following a reversal in the theoretical tick. This occurs because `get_drift_direction` determines the direction from changes in staking-rate ratios rather than the difference between the current strategy midpoint and the target tick. For example, if the theoretical tick increases from 0 to 100 while the movement cap is 5, the strategy midpoint advances only from 0 to 5, although the stored theoretical tick becomes 100. If the theoretical tick subsequently decreases to 90, the strategy selects `DriftDirection::Decreasing`, even though the target remains substantially above the current midpoint.

Consequently, `cap_strat_mid_tick_per_rebalance` applies the decreasing-direction cap only when the proposed midpoint is below the current midpoint of 5. Since the proposed midpoint remains above 5, the function returns it without enforcing the movement cap. The subsequent `drifting_in_right_direction` check requires the new midpoint to be strictly below 5 and therefore reverts with `DriftingOppositeDirection`.

Because the failed rebalance does not update the stored `Autodrift` state, subsequent attempts under the same conditions encounter the same rejection, preventing the midpoint from gradually converging toward the target. This can disrupt automated rebalancing and leave liquidity concentrated in a stale or out-of-range position until market conditions or strategy parameters change.

Remediation

Determine the required movement direction from the current midpoint toward the target tick, and apply the per-rebalance cap accordingly to ensure gradual convergence despite theoretical-tick reversals.

Patch

Resolved in PR#1036.

© 2026 Otter Audits LLC. All Rights Reserved.

Zero Thresholds Bypass Snapshot Deviation Checks LOW OS-KYV-ADV-01

Description

In `invest::handler_invest`, the snapshot-price deviation check is executed only when `snapshot_max_deviation_bps > 0`.

If both the strategy-level and global `max_deviation_from_snapshot_price_bps` values are zero, the check is skipped entirely, regardless of `cap_max_deviation_from_snapshot_price_bps` and `HARD_CAP_MAX_DEVIATION_FROM_SNAPSHOT_PRICE_BPS`.

Consequently, even when `global_config.ref_price_max_slot_ttl` is nonzero, the instruction validates the reference-price snapshot's freshness without enforcing any deviation limit against the current pool price. This allows investments to proceed despite discrepancies between the stored snapshot and the current pool price.

Remediation

Enforce a nonzero snapshot-price deviation threshold whenever `ref_price_max_slot_ttl` is enabled, reserving a zero TTL for disabling snapshot validation.

Patch

Resolved in PR#1023.

© 2026 Otter Audits LLC. All Rights Reserved.

Rounding Up Inflates Percentage-Based Investment Caps LOW OS-KYV-ADV-02

Description

In `invest_cap::compute_strategy_value_usd_for_invest_cap`, `underlying_inventory` uses `LiquidityCalculationMode::Deposit`, which rounds the underlying token amounts represented by invested liquidity upward for Orca and Raydium. This can slightly overestimate the strategy's `USD` value, resulting in a higher percentage-based investment cap than intended. Although the discrepancy is typically negligible dust, it may become significant for expensive tokens with low decimal precision.

>_ *programs/yvaults/src/operations/invest_cap.rs* RUST

```rust
fn compute_strategy_value_usd_for_invest_cap(
    strategy: &WhirlpoolStrategy,
    clmm: &dyn Clmm,
    prices: &StrategyTokenPrices,
) -> VaultResult<U128> {
    let (available, invested, fees) = common::underlying_inventory(
        strategy,
        clmm,
        SimulationPrice::PoolPrice,
        LiquidityCalculationMode::Deposit,
        clmm.get_position_liquidity()?,
    )?;
    [...]
}
```

Remediation

Use `LiquidityCalculationMode::Withdraw` to round invested token amounts downward and ensure a more conservative investment-cap calculation.

Patch

This issue was acknowledged by Kamino.

© 2026 Otter Audits LLC. All Rights Reserved.

Rounded Meteora Withdrawals Inflate Withdrawal Cap LOW OS-KYV-ADV-03

Description

In `vault_operations::withdraw`, withdrawals from Meteora convert the user's proportional share entitlement into `bps_to_disinvest`, rounded up to the nearest basis point to accommodate the CLMM's withdrawal precision.

However, the withdrawal caps are updated using `available_to_withdraw.a + delta_a` and `available_to_withdraw.b + delta_b` respectively, where `delta_a` and `delta_b` represent the full amounts removed from the CLMM rather than the amounts actually allocated to the user. The function subsequently scales these amounts by the ratio between the user's exact share percentage and the rounded-up withdrawal percentage, returning the excess to the strategy's available balances.

>_ *programs/yvaults/src/operations/vault_operations.rs* RUST

```rust
pub fn withdraw(
    strategy: &mut WhirlpoolStrategy,
    config: &GlobalConfig,
    clmm: &dyn Clmm,
    number_of_shares: u64,
    slippage: u64,
    curr_timestamp: i64,
) -> VaultResult<WithdrawEffects> {
    [...]
    add_to_withdrawal_accum(
        &mut strategy.withdrawal_cap_a,
        available_to_withdraw.a.checked_add(delta_a).unwrap(),
        curr_timestamp.try_into()?,
    )?;
    add_to_withdrawal_accum(
        &mut strategy.withdrawal_cap_b,
        available_to_withdraw.b.checked_add(delta_b).unwrap(),
        curr_timestamp.try_into()?,
    )?;
    [...]
}
```

Consequently, the withdrawal accumulators can substantially overstate the assets redeemed by users, particularly for small withdrawals. For example, 1,000 withdrawals of 0.01 bps each could individually trigger a 1 bps CLMM removal, consuming withdrawal capacity equivalent to 1,000 bps despite users collectively receiving only 10 bps of the invested assets.

© 2026 Otter Audits LLC. All Rights Reserved.

Remediation

Update withdrawal caps using `available_to_withdraw` plus `amounts_to_user`, excluding excess CLMM removal amounts recredited to the strategy.

Patch

This issue was acknowledged by Kamino.

© 2026 Otter Audits LLC. All Rights Reserved.

05 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-KYV-SUG-00	Pending rebalance ticks remain valid indefinitely, so delayed execution may open a range based on stale market conditions.
OS-KYV-SUG-01	The `DeprecatedAdminAuthority` check in `update_pubkey_config` is unreachable because `handler_update_global_config` already rejects the same key before dispatch.
OS-KYV-SUG-02	The `open_liquidity_position` instruction incorrectly blocks a global administrator when the signer also holds the local strategy-admin role and local administration is disabled.
OS-KYV-SUG-03	`update_reward_mapping`, `initialize_kamino_reward`, and `add_kamino_rewards` remain callable during emergency mode, allowing strategy state modifications despite the intended freeze.
OS-KYV-SUG-04	`single_token_deposit_with_min` can panic on insufficient token balances due to `unwrap()`, while `get_minting_method` silently accepts invalid minting-method values as proportional.
OS-KYV-SUG-05	Recommendations to improve overall clarity, functionality, and address discrepancies.
OS-KYV-SUG-06	When `reward_discount_bps` is unset, the fallback uses the more permissive strategy or global swap-slippage tolerance, potentially increasing the reward discount. Therefore, the comment incorrectly implies that the fallback preserves the previous incentive.

© 2026 Otter Audits LLC. All Rights Reserved.

OS-KYV-SUG-07	The default branch of `cap_strat_mid_tick_per_rebalance` returns the proposed midpoint without enforcing the movement cap, relying on subsequent validation.
OS-KYV-SUG-08	Certain combinations of reference-price minimum age and TTL can cause repeated snapshot refreshes, preventing the investment bot from progressing to the invest instruction.

© 2026 Otter Audits LLC. All Rights Reserved.

Stale Pending Ticks Enable Delayed Rebalances OS-KYV-SUG-00

Description

Pending rebalance ticks currently remain valid indefinitely, allowing a delayed `open_liquidity_position` call to execute a range selected under outdated market conditions. Operational delays caused by governance approval latency, bot downtime, or transaction failures may therefore leave stale ticks that no longer satisfy the strategy's intended pricing assumptions.

Remediation

Add an expiration timestamp to the pending range to require the instruction to reject obsolete rebalance proposals before opening a new position.

Patch

This issue was acknowledged by Kamino.

© 2026 Otter Audits LLC. All Rights Reserved.

Unreachable Admin Authority Validation OS-KYV-SUG-01

Description

The `DeprecatedAdminAuthority` branch in `global_config_operations::update_pubkey_config` is unreachable because its only caller, `handler_update_global_config`, already rejects this configuration key with `CannotDirectlyChangeAdminAuthority` before dispatching to the helper.

>_ *programs/yvaults/src/config/global_config_operations.rs* RUST

```rust
pub fn update_pubkey_config(
    global_config: &mut GlobalConfig,
    key: GlobalConfigOption,
    value: &Pubkey,
) -> Result<(), VaultError> {
    match key {
        [...]
        GlobalConfigOption::DeprecatedAdminAuthority => {
            return Err(VaultError::CannotDirectlyChangeAdminAuthority);
        }
        [...]
    }
    Ok(())
}
```

Consequently, the duplicate validation provides no additional protection and introduces unnecessary redundancy between the instruction handler and configuration logic.

>_ *programs/yvaults/src/instructions/update_global_config.rs* RUST

```rust
pub fn handler_update_global_config(
    ctx: Context<UpdateGlobalConfig>,
    key: u16,
    index: u16,
    value: &[u8; VALUE_BYTE_ARRAY_LEN],
) -> Result<()> {
    let global_config = &mut ctx.accounts.global_config.load_mut()?;
    let key =
        GlobalConfigOption::try_from(key).map_err(|_| ProgramError::InvalidInstructionData)?;
    msg!("update global config field {:?}", key);
    match key {
        [...]
        GlobalConfigOption::DeprecatedAdminAuthority => {
            Err(VaultError::CannotDirectlyChangeAdminAuthority.into())
        }
        [...]
    }
}
```

© 2026 Otter Audits LLC. All Rights Reserved.

Remediation

Consolidate the check into a single location to simplify the implementation and maintain a consistent validation path.

Patch

Resolved in PR#1032.

© 2026 Otter Audits LLC. All Rights Reserved.

Local Admin Restrictions Incorrectly Block Global Admin OS-KYV-SUG-02

Description

In `handler_open_liquidity_position`, the local-administrator blocking check immediately reverts with `LocalAdminBlocked` when the signer matches `strategy.admin_authority` and either the strategy-level or global local-admin block is active. However, this check does not account for a signer who is simultaneously the strategy administrator and `config.admin_authority`, causing the global administrator to be incorrectly denied access despite retaining higher-level privileges. This behavior is inconsistent with `update_strategy_config`, where `is_blocked_strategy_admin` explicitly excludes the global administrator.

Remediation

Exempt `config.admin_authority` from the local-admin blocking check in `handler_open_liquidity_position` to preserve global administrative privileges.

Patch

Resolved in PR#1014.

© 2026 Otter Audits LLC. All Rights Reserved.

Reward Instructions Bypass Emergency Mode Restrictions OS-KYV-SUG-03

Description

The `update_reward_mapping`, `initialize_kamino_reward`, and `add_kamino_rewards` instructions do not enforce emergency-mode restrictions, allowing strategy configuration and reward-related state to be modified while the strategy or protocol is frozen. This behavior is inconsistent with other strategy-mutating instructions.

Remediation

Enforce the relevant emergency-mode checks or, if these instructions are unused or unnecessary, remove them.

Patch

Resolved in PR#1014.

© 2026 Otter Audits LLC. All Rights Reserved.

Insufficient Validation of Invalid Input Values OS-KYV-SUG-04

Description

In `deposit::single_token_deposit_with_min`, both deposit amounts are calculated using `checked_sub().unwrap()`, which causes the instruction to panic if a requested minimum post-deposit balance exceeds the corresponding token-account balance. Replace `unwrap()` with `ok_or()` and a descriptive `VaultError` to return an explicit error instead of triggering a panic.

>_ *programs/yvaults/src/instructions/deposit.rs* RUST

```rust
pub fn single_token_deposit_with_min<'info>(
    ctx: Context<'_, '_, '_, 'info, Deposit<'info>>,
    token_a_min_post_deposit_balance: u64,
    token_b_min_post_deposit_balance: u64,
    min_shares_out: u64,
) -> Result<()> {
    [...]
    let token_a_to_deposit = token_a_ata_balance
        .checked_sub(token_a_min_post_deposit_balance)
        .unwrap();

    let token_b_to_deposit = token_b_ata_balance
        .checked_sub(token_b_min_post_deposit_balance)
        .unwrap();
    [...]
}
```

Also, in `vault_operations::get_minting_method`, any `share_calculation_method` value other than `MintingMethod::PriceBased` is implicitly treated as `MintingMethod::Proportional`, allowing invalid `enum` discriminants to pass without rejection. Explicitly match both supported variants and return an error for unsupported values to prevent unintended minting behavior.

Remediation

Incorporate the error handling logic stated above.

Patch

Resolved in PR#1059.

© 2026 Otter Audits LLC. All Rights Reserved.

Code Maturity OS-KYV-SUG-05

Description

1. In `autodrift`, configuring `max_ticks_per_rebalance` below the minimum effective midpoint movement permitted by the DEX's tick spacing can cause subsequent `rebalances` to fail with `AutodriftMaxTicksTooLowForTickSpacing`. Currently, this incompatibility is detected only during rebalance execution, potentially leaving the strategy unable to progress under the configured cap. Validate the cap against the DEX's tick spacing when updating rebalance parameters, rejecting incompatible values or issuing an explicit warning before applying the configuration.
2. `handler_change_pool` allows a strategy to migrate to a pool with different tick spacing without resetting its `Autodrift` state. Consequently, the next `rebalance` in `open_liquidity_position` may normalize the previously recorded midpoint and epoch anchor using the new pool's tick spacing, violating the assumption that stored `Autodrift` state corresponds to the current pool's tick configuration. Although this does not appear directly exploitable, it creates an inconsistency. Reset the strategy's `Autodrift` state in `handler_change_pool` whenever the new pool's tick spacing differs from the previous pool's.
3. Using `RebalanceAutodriftWindow` for `epoch_cap_window` introduces unnecessary fields and couples epoch-cap tracking to unrelated `Autodrift` state. Replace `epoch_cap_window` with a dedicated, minimal structure containing only the fields required for epoch-cap tracking, reducing unnecessary state overhead.

Remediation

Implement the above-mentioned suggestions.

Patch

Resolved in PR#1046.

© 2026 Otter Audits LLC. All Rights Reserved.

Reward Discount Fallback No Longer Preserves Incentive OS-KYV-SUG-06

Description

The comment above `state::get_reward_discount_bps` states that an unset reward discount preserves the existing incentive by falling back to vault-swap slippage.

>_ *programs/yvaults/src/config/global_config_operations.rs* RUST

```rust
// Use the most permissive strategy or global uneven-swap tolerance so either configuration can
// relax the bound. The hard cap remains the absolute ceiling.
pub fn get_swap_vault_slippage_bps(&self, global_config: &GlobalConfig) -> u64 {
    u64::from(self.swap_vault_max_slippage_bps)
        .max(global_config.min_swap_uneven_slippage_tolerance_bps)
        .min(MAX_SWAP_VAULT_SLIPPAGE_BPS)
}

/// Fraction of the reward market value that the caller of the permissionless `swap_rewards`
/// does not have to pay. An unset dedicated value preserves the existing incentive by falling
/// back to the effective swap-vault slippage.
pub fn get_reward_discount_bps(&self, global_config: &GlobalConfig) -> u64 {
    let configured_discount_bps = if self.reward_discount_bps == 0 {
        self.get_swap_vault_slippage_bps(global_config)
    } else {
        self.reward_discount_bps
    };
    configured_discount_bps.min(CAP_REWARD_DISCOUNT_BPS)
}
```

However, `state::get_swap_vault_slippage_bps` now selects the more permissive value between the strategy and global tolerances, potentially increasing the fallback reward discount. Consequently, the fallback no longer necessarily preserves the previous incentive, and the comment should be updated to reflect this behavioral change.

Remediation

Update the comment to clarify that the fallback uses the more permissive effective swap-slippage tolerance and may increase the reward discount.

© 2026 Otter Audits LLC. All Rights Reserved.

Default Branch May Bypass Midpoint Cap OS-KYV-SUG-07

Description

In `RebalanceAutodrift ::cap_strat_mid_tick_per_rebalance`, the default match branch returns `proposed` without applying any movement cap, relying on the caller to reject movements inconsistent with the expected drift direction. Although the current `DriftDirection` `enum` only contains `Increasing` and `Decreasing`, introducing additional variants or changing the caller's validation could allow unintended midpoint movements to bypass the cap.

>_ *programs/yvaults/src/utils/rebalance_rules/autodrift.rs* RUST

```rust
fn cap_strat_mid_tick_per_rebalance(
    &self,
    proposed_strat_mid_tick: i32,
    drift_direction: &DriftDirection,
) -> VaultResult<i32> {
    // The first rebalance after creation or a params reset has no trusted previous
    // strategy midpoint to cap against, so it is intentionally left uncapped.
    if self.parameters.max_ticks_per_rebalance == 0
        || !matches!(self.state.step, RebalanceAutodriftStep::Autodrifting)
    {
        return Ok(proposed_strat_mid_tick);
    }
    [...]
    let capped = match drift_direction {
        DriftDirection::Increasing if proposed > current => proposed.min(current + max_ticks),
        DriftDirection::Decreasing if proposed < current => proposed.max(current - max_ticks),
        _ => proposed,
    };
    Ok(i32::try_from(capped)?)
}
```

Remediation

Return an explicit error in the default branch of `cap_strat_mid_tick_per_rebalance` to prevent unintended movements from bypassing the midpoint cap.

Patch

Resolved in PR#1036.

© 2026 Otter Audits LLC. All Rights Reserved.

Reference Price Configuration Causes Investment Livelock OS-KYV-SUG-08

Description

Certain combinations of `min_ref_price_age_seconds` and `ref_price_max_ttl_seconds` can cause the investment bot to repeatedly refresh reference-price snapshots without successfully investing. For example, with a TTL of 100 seconds and a minimum age of 50 seconds, the bot waits 50 seconds for the snapshot to mature, followed by a randomized 15 to 30-second delay before attempting investment. By then, the snapshot exceeds the bot's early-refresh threshold of 55 (`max(100 - 45, 50)`) seconds, causing it to refresh the snapshot during a subsequent attempt and restart the minimum-age waiting period. This can prevent investments from progressing under the affected configuration, although administrators can restore liveness by adjusting the parameters.

Remediation

Validate reference-price TTL and minimum-age configurations against the bot's refresh buffer and execution delays to prevent repeated snapshot refreshes and investment livelocks.

Patch

Resolved in PR#1074.

© 2026 Otter Audits LLC. All Rights Reserved.

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

© 2026 Otter Audits LLC. All Rights Reserved.

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.

© 2026 Otter Audits LLC. All Rights Reserved.